

Target Allocation — คู่มือการทำงานของระบบ

ระบบกระจายเป้าหีบจากหัวหน้าทีมลงพนักงานขายรายคน แล้วส่งกลับเข้า Target Sun

เอกสารนี้อธิบาย **สภาพระบบ ณ ปัจจุบัน** หลังการแก้ไขรอบล่าสุด · ทุกหมายเลขบรรทัดและค่าคงที่ตรวจจาก source จริง · กฎเชิงลึกของการกระจายดูที่ `docs/ALLOCATION_INVARIANTS.md`

สารบัญ

1. ระบบนี้ทำอะไร

2. ข้อมูลเข้า-ออก

3. ขั้นตอนการใช้งาน 3 ขั้นตอน

4. เครื่องกระจายหีบ

5. กฎที่ระบบบังคับ

6. แก้มือและการเปลี่ยนอัตโนมัติ
7. โหมดรวมภาค

8. ส่งเข้า Target Sun

9. บทบาทและสิทธิ์

10. ไฟล์ใน data/ และการใช้พร้อมกัน

11. สวิตช์ที่ตั้งได้

12. API ทั้งหมด

13. การทดสอบ

1 · ระบบนี้ทำอะไร

โจทย์: Target Sun กำหนด เป้าหีบรวมต่อ **SKU** ของทั้งทีม มาให้ หัวหน้าทีม (Supervisor) ต้องแตกเป้านั้นลงไปว่า พนักงานขายคนไหน **ควรได้กี่หีบของสินค้าตัวไหน** แล้วส่งผลกลับเข้า Target Sun

ทำไมไม่ไหวเพราะทีมหนึ่งมี SKU ระดับ 200–650 รายการ × พนักงาน 5–30 คน = หลายพันช่อง และต้องพอใจสองเงื่อนไขพร้อมกัน:

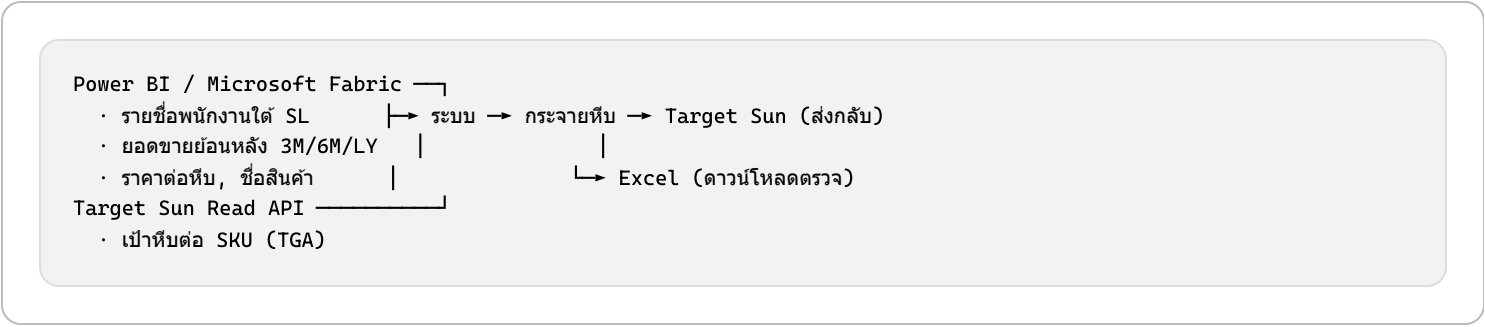
- หีบต่อ **SKU** ต้องครบเป้า ห้ามขาดห้ามเกิน
- มูลค่าเงินต่อคนต้องใกล้เคียงเป้าหมายรายคน (เป้าเหลือง / `yellow_target`)

ระบบแก้ให้ด้วยการผสม *สัดส่วนจากประวัติการขาย* กับ *Linear Programming* แล้วให้หัวหน้าปรับมือหีบได้ในจุดที่ต้องการ

ศัพท์ที่ใช้ตลอดเอกสาร

หีบ / box	หน่วยนับสินค้าที่กระจาย
เป้าหีบ	<code>supervisor_target_boxes</code> — เป้าต่อ SKU ของทั้งทีม มาจาก Target Sun
เป้าเหลือง	<code>yellow_target</code> — เป้าเงินรายคน (บาท)
SL / SuperCode	รหัสทีม เช่น <code>SL330</code>
baseline	จำนวนหีบที่คำนวณจากสัดส่วนประวัติ ใช้เป็นหลักยึด
ล็อก	ช่องที่ผู้ใช้พิมพ์แก้มือ ระบบห้ามขยับ

2 · ข้อมูลเข้า-ออก



แหล่งเป้าหีบสลับได้

ค่า `target_read_source` ใน `config/app_runtime.json` เลือกว่าจะอ่านเป้าจาก **Target Sun Read API** หรือ **Fabric** สลับได้จากหน้าแอดมิน ไม่ต้อง deploy

URL ของ Target Sun **ไม่ได้**อยู่ใน `.env` — ตั้งใน `backend/services/targetsun_endpoints.py` แล้วสลับ preset (`test` / `uat` / `prod` / `code`) จากหน้าแอดมิน ทำให้ทดสอบแบบ *อ่านจาก Prod ส่งเข้า UAT* ได้

ประวัติที่ใช้

ชุด	คือ	ใช้ตอน
<code>hist_cache_*.csv</code>	ยอดหีบ emp×sku ย้อนหลัง 3 เดือน	กลยุทธ์ L3M (ค่าเริ่มต้น)
<code>hist_cache*_6m.csv</code>	ย้อนหลัง 6 เดือน	กลยุทธ์ L6M
<code>hist_lysm_*.csv</code>	เดือนเดียวกันปีที่แล้ว	กลยุทธ์ LY · คอส์มัน์เทียบในตาราง
<code>hist_prev_*.csv</code>	เดือนก่อนหน้า	คอส์มัน์เทียบในตาราง
<code>hist_cy_*.csv</code>	ทั้งปีปฏิทิน (ปีนี้ + ปีที่แล้ว)	ตรวจว่า SKU ไหนเป็น "สินค้าใหม่"

3 · ขั้นตอนการใช้งาน 3 ขั้น

ขั้น 1 ข้อมูลตั้งต้น

เลือกทีม + งวด (เดือน/ปี) ระบบดึงพนักงาน เป้าหีบ ราคา และประวัติ แล้วเขียนลง cache ใน data/ ทุกไฟล์แยกตาม (ทีม, งวด)

ถ้างวดที่เลือกไม่มีเป้าใน TGA ระบบตอบ **409** และไม่ให้เข้าหน้าถัดไป

ขั้น 2 กำหนดเป้าเงินรายคน (ไม่บังคับ)

ปรับ เป้าเหลือง รายคนได้ ถ้าไม่แตะ ระบบใช้ค่าจาก Target Sun ตามเดิม มีช่องบันทึกเหตุผลกรณีเป้าโตติดลบ และหักลด BUI

ขั้น 3 กระจายหีบ

เลือกวิธี (เลือกได้มากกว่า 1 แล้วกำหนดว่าแบรนด์ไหนใช้วิธีไหน) กด **เริ่มคำนวณ** ได้ตารางผลลัพธ์ที่แก้ตัวเลขได้ แล้วดาวน์โหลด Excel หรือส่งเข้า Target Sun

4 · เครื่องกระจายหีบ

อยู่ใน `backend/OR_engine.py` — import แค่ `pandas` กับ `logging` ไม่แตะไฟล์ ไม่แตะเน็ต ไม่แตะฐานข้อมูล ทดสอบได้ในหน่วยความจำล้วน

วิธีกระจายที่เลือกได้

รหัส	หลักการ	ผ่าน LP?	เห็นใน UI?
L3M ค่าเริ่มต้น	สัดส่วนยอดขายเฉลี่ย 3 เดือนย้อนหลัง	ใช่	เห็น
L6M	สัดส่วนยอดขายเฉลี่ย 6 เดือน	ใช่	เห็น
LY	สัดส่วนเดือนเดียวกันปีที่แล้ว	ใช่	เห็น
PUSH	ผลักดันคนขายน้อย — ให้นำหนักกลับด้าน <code>max - hist</code>	ไม่	เห็น
EVEN	เฉลี่ยเท่ากันทุกคน	ไม่	ซ่อน
LP	ยึดเป้าเงินเป็นหลัก (baseline L3M)	ใช่	ซ่อน

ทั้ง 6 ตัวรองรับใน backend (`VALID_STRATEGIES`) แต่ **EVEN** และ **LP** ถูกคอมเมนต์ปิดไว้ใน `frontend/index.html` ชั่วคราว — UI จึงแสดงแค่ 4 ตัว ยิ่งผ่าน API ตรง ๆ ยังใช้ได้ทั้ง 6

ลำดับการทำงาน 6 ขั้น

1. `_normalize_engine_inputs`

2. `_proportional`

3. `_lp_optimize` (CBC)

4. `_greedy_revenue_balancer`

5. `_enforce_even_skus_on_df`

6. `validate_allocation_vs_targets`

ทำคีย์ให้อยู่รูปเดียวกัน · ปิดเป้าเป็น int · ยูน SKU ซ้ำ

· ปฏิเสธถ้าหีบที่ล็อกรวมกันเกินเป้า SKU

แบ่งตามสัดส่วนประวัติ + เพดานกันกระจุก ($\text{mean} \times 3.0$)

→ ได้ baseline ที่ทุกชั้นถัดไปใช้อ้างอิง

ปรับให้มูลค่าเงินต่อคนใกล้เคียง เป้า ภายในรั้ว $\pm\%$ จาก baseline

แก้ไม่ได้ → ใช้ผลข้อ 2 แทน (ตั้ง `optimization_fallback`)

โยนหีบที่ละใบระหว่างคนที่เกินขนาด ปิดช่องว่างที่เหลือ

สินค้าใหม่ที่ตึกไว้ → บังคับแบ่งเท่าทุกคน

ประตู่: ผลรวมต่อ SKU ต้องตรงเป้า ไม่งั้น 409

รั้งกันตัวเลขเพี้ยน

หัวใจคือ **baseline** จากประวัติ แล้วอนุญาตให้ LP ขยับได้แค่ในกรอบ %:

ประเภท SKU	รู้	ค่าคงที่
SKU หลัก (สะสมมูลค่าเข้าถึง 80% แบบ Pareto)	±35%	<code>_TIER_FLEX_BAND_PCT</code>
SKU รอง	±12%	<code>_TIER_STRICT_BAND_PCT</code>
ไม่เปิดโหมดหลัก/รอง	±20%	<code>_DEFAULT_HIST_BAND_PCT</code>
สินค้าใหม่ที่ดักแบ่งเท่า	ล็อกตายตามค่าแบ่งเท่า	—
คนที่ไม่มีประวัติ SKU นั้น (baseline 0)	เพดาน <code>ceil(เป้า ÷ คน × 3.0)</code>	ปิดเป็นค่าเริ่มต้น

ทำไมเพดาน baseline 0 ถึงปิดไว้

วัดกับข้อมูลจริง 8 ทีมแล้ว เปิดแล้วหีบย้ายที่ราว 2–3.5% ของทั้งทีม (ผลรวมต่อ SKU ยังตรงเป้าทุกกรณี แะย้ายจากคนไม่มีประวัติไปหาคนมีประวัติ) หัวหน้าทีมจะเห็นตัวเลขต่างทันที จึงถือเป็นการตัดสินใจเชิงนโยบาย ไม่ใช่บั๊ก

เปิดได้ด้วย `ALLOC_ZERO_BASELINE_CAP=1` · ตัวคูณขั้นแรก (เพดาน $\text{mean} \times 3$, anchor term ถ่วงด้วยราคา, สมการผลรวมต่อ SKU) ทำงานอยู่แล้วทั้งหมด

ค่าที่หน้าเว็บส่งมาจริง

พารามิเตอร์	ค่า	ความหมาย
<code>tiered_allocation</code>	<code>true</code>	เปิดโหมด SKU หลัก/รอง
<code>tier_pct</code>	<code>0.80</code>	SKU หลัก = สะสมมูลค่าเข้าถึง 80%
<code>hist_balance</code>	<code>0.35</code>	น้ำหนักยึด baseline ใน LP
<code>revenue_tolerance_baht</code>	<code>1000</code>	ยอมให้มูลค่าต่อคนคลาดเป้าได้เท่าไร

ข้อควรระวังสำหรับคนแก้โค้ด — ชื่อตัวแปร LP

ห้ามเอา `emp_id / sku` ไปต่อเป็นชื่อตัวแปร PuLP PuLP ล้างเฉพาะอักขระใน `"-+[] ->/"` ซึ่งแปลง `-` เป็น `_` ทำให้ `("A-B", "C")` กับ `("A", "B_C")` ได้ชื่อชนกัน → CBC พังเป็น `PulpSolverError` → ตกไป fallback แบบเสียบ ๆ ใช้ `_vname()` ที่สร้างชื่อจากเลขลำดับเท่านั้น

5 · กฎที่ระบบบังคับ

รายละเอียดเต็มอยู่ใน docs/ALLOCATION_INVARIANTS.md — ย่อไว้ที่นี่

#	กฎ	บังคับที่ไหน
I1	ผลรวมหีบต่อ SKU = เป้าหีบ เป๊ะ ๆ	/optimize ตอบ 409 · ไม่เขียนไฟล์ ไม่สร้าง Excel
I2	เซลล์ที่ล็อกออกมาตรงค่าที่ล็อก	คีย์ normalize ที่ประตูเข้า · ล็อกเกินเป้าตอบ 400
I3	หีบเป็นจำนวนเต็ม ไม่ติดลบ	ปิดเป็น int ครั้งเดียวที่ประตูเข้า
I4	force_min_one → ทุกคนได้ ≥ 1 หีบ (เมื่อเป้า $\geq$ จำนวนคน)	รอดผ่านทั้ง proportional → LP → greedy
I5	เซลล์อยู่ในรั้ว % จาก baseline	ข้อจำกัดใน LP + ขอบเขตใน greedy
I6	SKU รหัสเดียวถูกกระจายรอบเดียว	ยุบซ้ำก่อนแบ่งกลุ่มตามแบรนด์
I7	โหมดรวมภาคไม่ปน ตัวตน ข้ามทีม	คีย์ล็อกมีรหัสขุนาหน้า · ห้าม fallback ไปรหัสผู้จัดการ

6 · แก้มือและการเกลี่ยอัตโนมัติ

คลิกตัวเลขในตารางแล้วพิมพ์ใหม่ → ช่องนั้นกลายเป็น **ล็อก** ระบบจะไม่ขยับอีก แม้กดคำนวณใหม่ (ส่งไปเป็น `locked_edits`)

แต่ผลรวมต่อ SKU ต้องคงเดิม ระบบจึงไป **เกลี่ยจากคนที่ยังไม่ถูกแก้** โดยอัตโนมัติ (`autoRebalance`) นำหนักการเกลี่ยใช้ `hist_avg` ของแต่ละคน

แผง [ผลจากการแก้ล่าสุด]

ทุกครั้งที่แก้ จะมีแผงขึ้นใต้ตารางบอกว่าไปกระทบใครบ้าง:

ผลจากการแก้ล่าสุด

×

111294 ฟลอเรสบูชมัน 80 ก.
[คุณแก้] C431 สมชาย +1
[ระบบเกลี่ยจาก] C439 ชานนท์ SL460 -1
⚠️ ดึงข้ามทีม: SL460

ทำงานโดยถ่ายภาพก่อนแก้แล้ว diff หลังเกลี่ยเสร็จ จึงจับได้ทุกเส้นทาง ไม่ต้องพึ่งตัวเกลี่ยรายงานเอง

7 · โหมดรวมภาค

ผู้จัดการ (หรือซัพที่มี peer ในภาคเดียวกัน) เปิดดูหลายทีมพร้อมกันในตารางเดียวได้ และกระจายหีบได้ทั้งภาค

สิ่งที่ตึง กับ สิ่งที่ยับได้ — ต่างกันตามจังหวะ

จังหวะ	ขอบเขตที่ถูกตึง	เพราะอะไร
กดคำนวณ / คำนวณใหม่	ต่อทีม	แต่ละซัพยัง /optimize แยก ใช้เป้าของตัวเอง + ประตุ 409 บังคับ
แก้มือแล้วระบบเกลี่ยต่อ	ทั้งภาค	autoRebalance จัดกลุ่มด้วย sku ล้วน เทียบกับ ผลรวมเป้าทั้งภาค

ผลคือหลังแก้มือ ผลรวม SKU ทั้งภาค ยังตรง แต่สัดส่วน รายทีม เลื่อนจากเป้า TGA ของทีมนั้นได้ — เป็นพฤติกรรมที่ตั้งใจ

แถวรวมทีม — จุดเดียวที่มองเห็นการเลื่อน

ท้ายกลุ่มของแต่ละซัพมีแถบสรุปบอกยอดต่อ SKU และส่วนต่างจากเป้าของทีมนั้น

- +3 สลัม = ทีมนี้ได้เกินเป้าตัวเอง 3 หีบ
- 3 สิ้นน้ำเงิน = ได้น้อยกว่าเป้าตัวเอง 3 หีบ
- [ตรงเป้าทีมทุก SKU] = ยังไม่มีการย้ายข้ามทีม

เป้ารายทีมอ่านจาก `target_boxes_by_sup` ใน payload รวมภาค ไม่ใช่ยอดบวกรวมทั้งภาค

ตารางรวมภาคไม่ได้มาจากการกระจายสดเสมอไป

ตอนเปิดหน้ามา ระบบประกอบตารางจากสองแหล่งต่อทีม:

- ทีมที่เคยกระจาย → **snapshot** เก็บ ที่บันทึกไว้ตอนนั้น
- ทีมที่ยังไม่เคยกระจาย → ตัวเลขจาก **Target Sun** ตรง ๆ

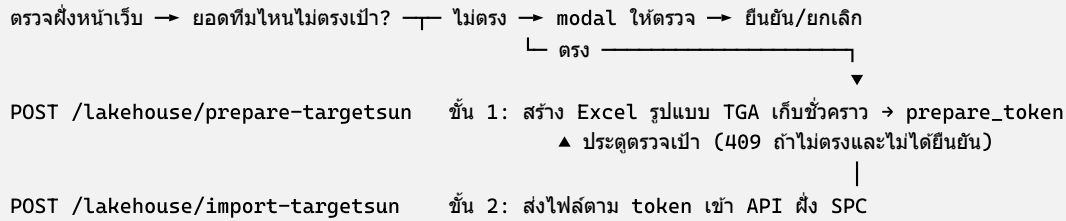
แถวรวมทีมจึงแสดงป้ายต่างกันตามแหล่ง — ทีมที่มาจาก Target Sun จะขึ้นว่า [ยังไม่ได้กระจาย] และ **ไม่แสดง +/-** เพราะเทียบกันไม่ได้

อื่น ๆ ที่ต่างจากมุมมองทีมเดียว

- โหลดขนานกัน — `ThreadPoolExecutor` 6 thread (เพดาน 8) ปรับด้วย `AGGREGATE_LOAD_WORKERS`
- ทีมพังกทีมเดียวไม่ลัมทั้งภาค — เก็บ error รายทีม รายงานว่าทีมไหนพลาด ทีมที่สำเร็จบันทึกแล้ว
- ค่าเตือน **fallback** รวมทุกทีม — ถ้าทีมใดตกไปใช้ proportional จะเตือนพร้อมระบุชื่อทีม

- คีย์ล็อกมีรหัสระบุหน้า (SL330::E001) เพราะ emp_id ซ้ำข้ามทีมได้

8 · ส่งเข้า Target Sun



ชั้นการตรวจ

จุด	ตรวจอะไร	ผ่านได้อย่างไร
/optimize	ผลรวมต่อ SKU ต่อทีม ทันทีหลังกระจาย	ต้องตรงเสมอ
PUT /data/allocations	ไม่ตรวจ	— (ตั้งใจ ให้แก็ระหว่างทางได้)
ดาวน์โหลด Excel	ไม่ตรวจ โดยตั้งใจ	ต้องโหลดมาตรวจได้เสมอแม้ตัวเลขยังไม่ตรง
ส่ง Target Sun	ผลรวมต่อ SKU ต่อทีม เทียบเป้าปัจจุบัน	ผู้ใช้กดยืนยันหลังเห็นรายการ

สำหรับคนแก่ได้

`_build_tga_upload_dataframe()` ถูกใช้ร่วมกันทั้งการสร้างไฟล์ให้ **ดาวน์โหลด** และการ **ส่งจริง** ประตูลังคคุมด้วยพารามิเตอร์ `enforce_targets` (ค่าเริ่มต้น `False`) และเปิดเป็น `True` เฉพาะใน `targetsun_import.py` เท่านั้น — ห้ามย้ายการตรวจเข้าไปในตัวสร้างแบบไม่มีเงื่อนไข ไม่งั้นยังตัวเลขมีปัญหา ยิ่งโหลดไฟล์มาตรวจไม่ได้

รูปแบบไฟล์ที่ส่ง

Excel ชีตเดียวชื่อ TGA คอลัมน์ตาม tga_target_salesman_next :

PRODUCTCODE · SALESTYPE · DIVISIONCODE · SALESMANCODE · AREACODE
PROVINCECODE · WAREHOUSECODE · QUANTITYCASE · EFFECTIVEDATE · UPDATEDATE · USERCODE

แถว QUANTITYCASE = 0 ถูกส่งไปด้วย **โดยตั้งใจ** เพื่อทับเป้าหมายเดิมใน Target Sun ให้เป็นศูนย์ · วันที่เป็นข้อความปี พ.ศ. รูปแบบ d/M/yyyy
HH:mm:ss

9 · บทบาทและสิทธิ์

สิทธิ์มาจาก `config/user_access.json` (อีเมล + รหัส USERPL) และลำดับชั้นใน `config/access_hierarchy.json` ล็อกอินผ่าน Microsoft Entra

บทบาท	เห็นอะไร	ทำอะไรได้
supervisor	ทีมตัวเอง	กระจาย · แก้ · บันทึก · ส่ง
supervisor_acc (region peers)	ทุกทีมใน div+ภาค+หน่วยเดียวกัน	แก้และกระจายทีมอื่นในกลุ่มได้
regional_manager	ทีมในภาคตัวเอง	รายคน + รวมภาค (กระจายได้)
manager_acc (division)	ทั้ง division	รวมทั้งหมด = ดูอย่างเดียว · ต้องเลือกรายคนหรือรวมภาคจึงกระจายได้
marketing	ตามที่กำหนด	เข้าบางส่วนของหน้าแอดมิน

สิทธิ์ส่ง Target Sun

คุมแยกต่างหากด้วย `can_import_targetsun` ต่ออีเมล ตั้งจากหน้าแอดมิน

คอลัมน์ Target Sun มีปุ่ม **เปิดทุกคน / ปิดทุกคน** ทำงานกับ **แถวที่แสดงอยู่ตอนนั้น** (กรองภาค/หน่วย/บทบาทก่อนได้) ถ้ามียืนยันพร้อมบอกจำนวนที่เปลี่ยนจริง และบันทึกลง usage log ทุกครั้ง

10 · ไฟล์ใน data/ และการใช้พร้อมกัน

ทุกไฟล์แยกตาม (ทีม, งวด) — ห้ามสร้างไฟล์ที่ไม่มีรหัสทีมในชื่อ

target_boxes_{SL}_{YYYY}_{MM}.csv	เป้าหมายราย SKU
target_sun_{SL}_{YYYY}_{MM}.csv	เป้าหมายราย emp
emp_cache_{SL}_{YYYY}_{MM}.csv	รายชื่อพนักงาน
hist_cache_{SL}_{YYYY}_{MM}.csv	ประวัติ 3M (+ _6m)
hist_cy_{SL}_{YYYY}.csv	ประวัติทั้งปี - ใช้ตรวจสินค้าใหม่
tga_lines_{SL}_{YYYY}_{MM}.csv	TGA grain เดิม (ใช้ตอนส่ง)
payload_cache_{SL}_{YYYY}_{MM}.json	cache ผล GET /data/employees
final_allocation_{SL}_{YYYY}_{MM}.csv	ผลกระจายล่าสุด
Final_Dashboard_{SL}_{YYYY}_{MM}.xlsx	Excel ผลกระจาย
allocations/{SL}_{YYYY}_{MM}.json	snapshot บน server

ข้อสมมติที่ทั้งระบบแบกอยู่: **uvicorn 1 worker** เท่านั้น

route handler ทุกตัวเป็น `def` ไม่ใช่ `async def` → FastAPI โยนเข้า `anyio threadpool (40 threads)` → **request** หลาย
คนรันขนานกันจริงในโปรเซสเดียว ระบบจึงใช้ `threading.Lock` ป้องกัน

เพิ่มเป็น **2 worker** หรือขึ้นหลาย **container** เมื่อไหร่ **lock** ทุกตัวไร้ผลทันที เพราะเป็น lock ระดับโปรเซส ต้องเปลี่ยนไปใช้ `file lock` หรือฐานข้อมูลก่อน — ไม่ใช่แค่เพิ่มตัวเลข worker

การบันทึกผลกระจาย — **optimistic concurrency**

```
client โหลด → เห็น version: 3
client บันทึก → ส่ง if_match_version: 3
server: version บนดิสก์ == 3? → เขียน version: 4 → 200
      ≠ 3? → 409 พร้อมบอก version ปัจจุบัน
```

ใช้ `version` เป็นเลขจำนวนเต็ม ไม่ใช่ `updated_at` เพราะ `autosave debounce 800ms` ทำให้สอง `save` ในวินาทีเดียวกันได้
`timestamp` เท่ากัน = precondition มีรู้ให้เล็ดลอด

11 · สวิตช์ที่ตั้งได้

ตัวแปร	ค่าเริ่มต้น	ผล
AGGREGATE_LOAD_WORKERS	6	thread โหลดรวมภาค (เพดาน 8) · 1 = ที่ละหีมแบบเดิม
EMPLOYEE_PAYLOAD_CACHE_TTL_SEC	3600	อายุ cache ชั้น 1 · 0 = ปิด (อิง DAX ทุกครั้ง)
ALLOC_ZERO_BASELINE_CAP	ปิด	เปิดเพดานคนไม่มีประวัติ — ตัวเลขขยับ 2–3.5%
ALLOC_ALLOW_MISMATCH	ปิด	ฉุกเฉินเท่านั้น — ปลอยผลที่ไม่ตรงเป้าให้ผ่านทั้ง optimize และตอนส่ง
ALLOC_REQUIRE_IF_MATCH	ปิด	บังคับ client ส่ง version ตอนบันทึก (กัน tab เก่าเขียนทับ)
USE_LEGACY_TARGET_CSV	ปิด	dev เท่านั้น — อ่านไฟล์เป้าแบบ global ที่วางเอง
ENABLE_DEBUG_ENDPOINTS	ปิด	เปิด GET /debug/fabric

12 · API ทั้งหมด

ข้อมูลและการกระจาย

Method	Path	หน้าที่
GET	/managers	Dropdown ทีม/ผู้จัดการ
GET	/data/employees	Payload ชั้น 1 ของทีมเดียว
GET	/data/employees/aggregate	รวมหลายทีม (ผู้จัดการ) — โหลดขนาน
GET	/data/employees/region-peers	รวมทุกทีมในภาค (ซูป peer)
GET	/data/targets/live	ดึงเป้าล่าสุดจาก Target Sun
POST	/optimize	กระจายหีบ · 400 ล็อกเกินเป้า · 409 ผลรวมไม่ตรง

บันทึกผลและส่งออก

Method	Path	หน้าที่
GET/PUT/DELETE	/data/allocations	snapshot ผลกระจายบน server (CAS ด้วย version)
GET	/data/allocations/summary	สรุปสถานะทุกทีมที่เข้าถึงได้
POST	/export/excel · GET /download/excel	Excel สรุป Dashboard
POST	/lakehouse/export-csv	CSV รูปแบบ TGA
POST	/lakehouse/prepare-targetsun	ชั้น 1 ของการส่ง — สร้างไฟล์ + token
POST	/lakehouse/import-targetsun	ชั้น 2 — ส่งเข้า Target Sun
POST	/lakehouse/upload	อัปโหลดเข้า OneLake (API เท่านั้น ไม่มีปุ่มใน UI)

แอดมิน

Method	Path	หน้าที่
GET/POST/PUT/DELETE	/admin/user-access	จัดการสิทธิ์ผู้ใช้
PUT	/admin/user-access/targetsun	เปิด/ปิดสิทธิ์ส่ง ทิละอีเมล
PUT	/admin/user-access/targetsun/bulk	เปิด/ปิดยกชุด
GET/POST/PUT/DELETE	/admin/sl-links · /admin/sku-links	ผูกรหัส SL / รหัส SKU
PUT	/admin/settings/target-source	สลับแหล่งเป้า + preset URL
GET	/admin/usage-logs	บันทึกการใช้งาน
GET	/health · /health/build	ตรวจสอบสถานะ

13 · การทดสอบ

```
# ทั้งหมด (161 เทสต์)
python run_tests.py

# กฎคงที่ของการกระจาย - ในหน่วยความจำล้วน ไม่แตะไฟล์/เน็ต/DB
python -m pytest tests/test_or_engine_invariants.py -q

# ก่อน/หลังทำอะไรที่อาจทำให้ตัวเลขขยับ
python scripts/golden_allocation.py capture
python scripts/golden_allocation.py compare -v
```

สิ่งที่ต้องรู้เกี่ยวกับ golden harness

- **CBC ไม่ deterministic 100%** — มี time limit สูงสุด 60 วินาทีต่อการแก้หนึ่งครั้ง ทีมที่ SKU เยอะพอจะชน limit จะได้คำตอบ "ดีที่สุดเท่าที่หาห็น" รันซ้ำอาจต่าง 1-2 เคส
- **data/** เป็น **cache** ที่แอปเขียนทับได้ตลอด เป้าจึงเปลี่ยนโดยไม่มีใครแก้โค้ด harness เก็บลายนิ้วมือข้อมูลเอาไว้ ถ้าต่างจะรายงานเป็น [ข้อมูลต้นทางเปลี่ยน] แยกออกมา
- **sku_mismatches** ต้องเป็น **0** เสมอ ไม่ว่ากรณีไหน — ถ้าไม่ใช่คือกฎ l1 พัง

`run_tests.py` ถ่าย snapshot ไฟล์ config สำคัญก่อนรัน ถ้าเทสต์ไปแก้ไฟล์จริงจะ **กู้คืนอัตโนมัติ + เตือน + exit 1** (เคยพบว่า `config/app_runtime.json` ถูกเปลี่ยน preset ปลายทาง Target Sun ระหว่างรันเทสต์)

เอกสารประกอบ: `docs/ALLOCATION_INVARIANTS.md` (กฎการกระจายฉบับเต็ม) · `docs/CONCURRENCY.md` (การใช้พร้อมกัน) · `docs/DATA_FLOW.md` (แหล่งข้อมูล) · `docs/DEPLOY_QA_CHECKLIST.md` (ตรวจก่อน go-live) · `docs/user-manual-th.html` (คู่มือผู้ใช้)